

Introdução à Computação

escopo de variáveis e arrays

Alexandre Rademaker

variáveis e escopo I

Escopo é uma característica de uma variável que define a partir de quais funções essa variável pode ser acessada.

As variáveis **locais** só podem ser acessadas dentro das funções nas quais são criadas.

Variáveis **globais** podem ser acessadas por qualquer função no programa.

variáveis e escopo II

```
1 int triple(int x)
2 {
3     return x * 3;
4 }
5
6 int main(void)
7 {
8     int result = triple(5);
9 }
```

`x` é local na função `triple` e `result` é local na `main` .

variáveis e escopo III

```
1 #include <stdio.h>
2
3 float global = 0.5050;
4
5 void triple(void)
6 {
7     global *= 3;
8 }
9
10 int main(void)
11 {
12     triple();
13     printf("%f\n", global);
14 }
```

variáveis e escopo IV

Na maioria das vezes, as variáveis locais em C são passadas por valor nas chamadas de função.

Quando uma variável é passada por valor, a função **chamada** recebe uma cópia da variável passada, não a própria variável.

Isso significa que a variável no **chamador** permanece inalterada, a menos que seja substituída.

variáveis e escopo V

O que será impresso? 4 ou 12?

```
1 int triple(int x) {  
2     return x *= 3;  
3 }  
4  
5 int main(void) {  
6     int foo = 4;  
7     triple(foo);  
8     printf("%d", foo);  
9 }
```

```
1 int triple(int x) {  
2     return x *= 3;  
3 }  
4  
5 int main(void) {  
6     int foo = 4;  
7     foo = triple(foo);  
8     printf("%d", foo);  
9 }
```

variáveis e escopo VI

As coisas podem ficar particularmente enganadora se o mesmo nome de variável aparecer em várias funções, o que é perfeitamente aceitável, desde que as variáveis existam em escopos diferentes.

variáveis e escopo VII

```
1 int main(void)
2 {
3     int x = 1;
4     int y;
5     y = increment(x);
6     printf("x is %i, y is %i\n", x, y);
7 }
8
9 int increment(int x)
10 {
11     x++;
12     return x;
13 }
```

x is 1, y is 2

arrays I

Um array é um bloco de espaço contíguo na memória
que foi particionado em pequenos blocos de espaço de tamanho
idêntico chamados elementos
cada um dos quais pode armazenar uma certa quantidade de dados
todos do mesmo tipo de dados, como `int` ou `char`
e que pode ser acessado diretamente por um índice.

arrays II

Em C, os n elementos de um array são indexados a partir de 0. O último elemento estará na posição $n - 1$

C é muito indulgente. Nada irá prevenir que você **saia dos limites** de seu array; tome cuidado!

arrays III

Declaração

```
type name[size];
```

Exemplo

```
int student_grades[30];
```

arrays IV

you can perform the same operations with an element of the array that you would with variables of the same type.

```
1 bool truthtable[10];  
2  
3 truthtable[2] = false;  
4  
5 if(truthtable[7] == true) {  
6     printf("TRUE!\n");  
7 }  
8 truthtable[10] = true;
```

arrays V

Para declarar e inicializar um array, temos um “açúcar sintático”!

```
1 // instantiation syntax
2 bool truthtable[] = { false, true, true };
3
4 // individual element syntax
5 bool truthtable[3];
6 truthtable[0] = false;
7 truthtable[1] = true;
8 truthtable[2] = true;
```

arrays VI

Arrays podem ter mais do que uma dimensão!

```
bool battleship[10][10];
```

Na memória, de fato, apenas um array de 100 dimensões.

arrays VII

Embora possamos tratar elementos individuais dos arrays como variáveis, não podemos tratar arrays inteiros como variáveis.

Não podemos, por exemplo, atribuir um array a outro usando o operador de atribuição.

Em vez disso, devemos usar um loop para copiar os elementos um de cada vez.

```
int foo[5] = { 1, 2, 3, 4, 5 };  
int bar[5];  
bar = foo;
```

arrays VIII

```
int foo[5] = { 1, 2, 3, 4, 5 };
```

```
int bar[5];  
for(int j = 0; j < 5; j++)  
{  
    bar[j] = foo[j];  
}
```


arrays IX

Lembre-se que variáveis em C são passadas **por valor** nas chamadas de função.

Arrays não seguem esta regra. Eles são passados **por referência**. A função chamada recebe o array, não uma cópia dele.

O que isso significa quando a função chamada manipula elementos do array?

Por enquanto, apenas vamos entender que arrays têm essa propriedade especial, mas voltaremos ao assunto em breve!

arrays X

```
1 void set_array(int array[4]) {
2     array[0] = 22;
3 }
4
5 void set_int(int x) {
6     x = 22;
7 }
8
9 int main(void)
10 {
11     int a = 10; int b[4] = { 0, 1, 2, 3 };
12     set_int(a);
13     set_array(b);
14     printf("%d %d\n", a, b[0]); // prints 10, 22
15 }
```